CI/CD Pipeline — Complete Cheat Sheet

Continuous Integration & Continuous Delivery/Deployment · Real-World Scenario · Full Keyword Glossary

DevOps	Automation	Software Delivery	Best Practices	Real Scenario

Pipeline Overview — All 7 Stages

📂 Source Code	🔨 Build Compile	✅ Test Verify	🔒 Security Scan	📦 Artifact Package	🚀 Deploy Release	📊 Monitor Observe

Core Definitions — CI vs CD vs Continuous Deployment

Continuous Integration (CI)

What: Developers frequently merge code to shared repo. Each push triggers automated build + test.

Goal: Catch bugs early, keep codebase working, avoid integration hell.

```
git push origin feature/login
→  pipeline runs automatically
```

Continuous Delivery (CD)

What: Software always deployable. Production release needs manual approval.

Why: Teams release on demand, not on a quarterly fear-driven schedule.

Staging: auto-deployed | Production: manager approves

Continuous Deployment

What: Every change passing tests is auto-deployed to production — zero clicks.

Who: Teams with very high test coverage. Netflix/Amazon deploy thousands/day.

Merge to main → live in production in <15 minutes

Real-World Scenario — 'ShopEasy' E-Commerce App

Scenario: Developer Sara adds a Discount Code feature | React + Node.js + PostgreSQL | AWS EKS | GitHub Actions

1 — Stage 1 · Source Control
Sara creates a feature branch and opens a Pull Request

Sara branches off main into feature/discount-codes. She commits her code and opens a PR on GitHub. A webhook fires the moment she pushes, triggering the pipeline. Her teammate reviews the code — human quality gate before merge.

```
git checkout -b feature/discount-codes
git commit -m 'feat: add discount code
validation at checkout'
git push origin feature/discount-codes
# webhook triggers pipeline immediately
```

Result: Pipeline triggered successfully

2 — Stage 2 · Build
GitHub Actions runner installs dependencies and compiles

A cloud runner checks out the code. Runs npm ci (clean install from package-lock.json), TypeScript compiler to catch type errors, bundles React app, then builds a Docker image tagged with the git SHA for full traceability.

```
npm ci                              # exact
locked deps
npx tsc --noEmit                    #
TypeScript type check
npm run build                       # bundle
React app
docker build -t shopeasy:a3f9b2c .  #
containerise
```

Result: Build passed · Docker image created

3

Stage 3 · Test

Three layers of automated tests — unit, integration, E2E

Unit tests (Jest): 12 tests for discount validation — valid codes, expired, wrong format. Fast (0.8s). Integration tests: API against a real test PostgreSQL DB. E2E tests (Playwright): headless browser visits checkout, enters SAVE10, asserts 10% price drop. Coverage: 94%.

```
npm run test:unit         # 12 tests · 0.8s
npm run test:integration  # 5 tests · 4.2s
npm run test:e2e          # 3 flows · 22s
# Coverage: 94% on new code
```

Result: All 20 tests passed

4

Stage 4 · Security Scan

Snyk scans dependencies; SonarQube analyses source code

SCA (Snyk): checks 847 npm packages for CVEs — flags lodash prototype pollution (non-critical). SAST (SonarQube): finds SQL injection risk in Sara's code — BLOCKER. Pipeline fails. Sara fixes using parameterised queries, re-pushes, and the scan passes.

```
snyk test                   # check deps for
CVEs
sonar-scanner               # static code
analysis
# FAIL: SQL injection in
discountController.js:47
# Sara fixes → re-pushes → passes
```

Result: Security scan passed after SQL injection fix

5

Stage 5 · Artifact & Registry

Docker image tagged and pushed to Amazon ECR

The verified Docker image is pushed to Amazon ECR (private registry) tagged with git SHA and semver. Old images stay in registry for rollbacks — if v2.4.1 breaks production, redeploy v2.4.0 in seconds. Registry is immutable — nobody overwrites existing tags.

```
docker tag shopeasy:a3f9b2c \
  123456.ecr.aws/shopeasy:2.4.1
docker push 123456.ecr.aws/shopeasy:2.4.1
# stored · immutable · ready for deploy
```

Result: Image v2.4.1 published to ECR successfully

6

Stage 6 · Deploy

ArgoCD canary: 5% → 25% → 50% → 100% over 30 minutes

Staging: ArgoCD detects new image tag in Git (GitOps) and syncs to staging K8s cluster. Production canary: 5% of live traffic to v2.4.1. After 10 min of healthy metrics, traffic shifts to 25% → 50% → 100%. If errors spike, ArgoCD auto-rolls back to v2.4.0 in <2 min.

```
# staging: automatic ArgoCD GitOps sync
# production canary rollout:
kubectl argo rollouts set image shopeasy \
  shopeasy=123456.ecr.aws/shopeasy:2.4.1
# 5% → 25% → 50% → 100% over 30 min
```

Result: v2.4.1 live · 0 errors · canary rollout complete

7

Stage 7 · Monitor & Observe

Prometheus + Grafana tracks the new feature in real time

Grafana dashboard shows p99 latency and error rate for the discount endpoint. Alert: if error rate >1% for 5 min, PagerDuty notifies on-call engineer. Distributed tracing (Jaeger) reveals a 200ms slow query Sara optimises next sprint. DORA: lead time dropped to 47 minutes.

```
# Prometheus alert rule:
- alert: DiscountEndpointErrors
  expr: rate(http_errors[5m]) > 0.01
  for: 5m
  → PagerDuty notify on-call engineer
```

Result: p99: 42ms · error rate: 0.0% · lead time: 47min

Pipeline Config — GitHub Actions (Annotated)

```
name: ShopEasy CI/CD
on:
  push:
    branches: [main, 'feature/**']    # trigger on any branch push
  pull_request:                        # also trigger on PR open
env:
  IMAGE: 123456.ecr.aws/shopeasy       # no hardcoded secrets here!
jobs:
  build-test:
    runs-on: ubuntu-latest             # fresh VM for every run
    steps:
      - uses: actions/checkout@v4      # clone repo at this SHA
      - run: npm ci                    # reproducible install
      - run: npm run lint              # catch style issues early
      - run: npm test -- --coverage    # unit + integration tests
      - run: npm run build             # compile TypeScript / bundle
  security:
    needs: build-test                  # only runs if build-test passes
    steps:
      - run: snyk test --severity-threshold=high
      - run: sonar-scanner
  publish:
    needs: security
    if: github.ref == 'refs/heads/main'   # publish from main only
    steps:
      - run: docker build -t $IMAGE:${{ github.sha }} .
      - run: docker push  $IMAGE:${{ github.sha }}
  deploy-staging:
    needs: publish
    steps:
      - run: argocd app set shopeasy-staging --helm-set image.tag=${{ github.sha }}
```

Complete Keyword Glossary — Every Term Explained

VERSION CONTROL & BRANCHING

Git	Distributed version control tracking every file change. Each developer has the full history locally. *Example: git log shows all commits with author + timestamp*
Branch	Independent line of development. Branch off main, work safely, merge back when done. *Example: feature/discount-codes is isolated from main*
Commit	Snapshot of changes saved to Git history with a unique SHA hash. Atomic unit of version control. *Example: git commit -m 'fix: null pointer in checkout'*
Pull Request (PR)	Proposal to merge one branch into another. Enables code review and automatic pipeline checks. *Example: Also called Merge Request (MR) in GitLab*
Webhook	HTTP callback GitHub sends to your CI server instantly when an event happens (push, PR open). *Example: This is how pipelines get triggered automatically*
SHA / Commit hash	Unique 40-character fingerprint (e.g. a3f9b2c) identifying an exact commit. *Example: Used to tag Docker images — trace exactly what code is deployed*
Trunk-based dev	Everyone commits to main frequently (daily) using very short-lived branches. *Example: Branches live less than 1 day — preferred for fast CI/CD*
GitFlow	Branching strategy with long-lived develop, release, and hotfix branches. *Example: Suited for scheduled releases; less ideal for continuous deployment*

PIPELINE STRUCTURE

Pipeline	Full automated sequence of stages (source→build→test→deploy) that runs on every code change. *Example: Defined as code in a YAML file committed to the repo*
Stage	Logical phase in the pipeline. Stages run in order; a failed stage stops everything. *Example: Build, Test, Deploy are common stage names*
Job	Unit of work within a stage. Multiple jobs in the same stage can run in parallel. *Example: unit-tests and lint run as two parallel jobs*
Step	A single command or action inside a job. *Example: run: npm test is one step*
Runner / Agent	The machine (VM or container) that actually executes pipeline jobs. *Example: GitHub provides cloud runners; you can self-host too*
Trigger	The event that starts the pipeline — push, PR, scheduled cron, or manual button. *Example: on: push in GitHub Actions YAML*
Needs / depends-on	Declares that one job must complete before another starts. Creates execution order. *Example: deploy-staging needs: publish*
Matrix build	Run the same job across multiple configurations in parallel. *Example: Test on Node 18, 20, and 22 simultaneously*
Cache	Save downloaded dependencies between runs to speed up pipelines. *Example: Caching node_modules saves 2-3 minutes per run*
Secret / Env var	Sensitive values stored encrypted in CI settings and injected at runtime. *Example: Never put API keys or passwords in YAML files*
Idempotent	Running the same pipeline step multiple times produces the same result. *Example: Deploy scripts must be safe to re-run without side effects*
Fail fast	Design principle: cheapest checks run first so failures are caught early and cheaply. *Example: Lint and unit tests run before expensive E2E tests*

BUILD & PACKAGING

npm ci	Install Node.js deps from package-lock.json exactly — no version drift. Faster, reproducible. *Example: Use in CI; use npm install locally for development*
Lint	Static code style analysis — catches formatting errors, unused variables, anti-patterns. *Example: Tools: ESLint, Pylint, RuboCop*

Static analysis	Analyse source code without executing it to find bugs and security issues. *Example: Tools: SonarQube, Pylance, Checkstyle*
Docker image	Lightweight package containing app + runtime + dependencies. Runs identically everywhere. *Example: Same image on laptop, CI runner, and production server*
Dockerfile	Recipe text file defining how to build a Docker image step by step. *Example: FROM node:20 · COPY . . · RUN npm ci · CMD node app.js*
Semver	Semantic Versioning: MAJOR.MINOR.PATCH. v2.4.1 = breaking 2, new-feature 4, bugfix 1. *Example: Standard way to version software releases*
Immutable artifact	Once built and tagged, an artifact never changes. v2.4.1 always means the same image. *Example: Prevents 'works on my machine' surprises in production*
Reproducible build	Same source + same inputs = identical output every time, on any machine. *Example: Achieved by pinning exact dependency versions (lock files)*

TESTING

Unit test	Tests a single function in isolation. Dependencies replaced with mocks. Very fast (ms). *Example: Tools: Jest, JUnit, Pytest — you should have thousands*
Integration test	Tests multiple components working together — e.g. API hitting a real test database. *Example: Slower than unit tests but catches wiring bugs between components*
E2E test	End-to-End: headless browser simulates a real user clicking through the full application. *Example: Tools: Cypress, Playwright, Selenium*
Test coverage	The % of your code executed during tests. 80%+ is a common minimum target. *Example: High coverage doesn't guarantee quality — assertions must be meaningful*
Mock / Stub	Fake replacement for a real dependency (database, API) in unit tests. *Example: Controls what the dependency returns so you test your code, not theirs*
TDD	Test-Driven Development — write the test before the code. Red → Green → Refactor. *Example: Leads to better-designed, well-tested, maintainable code*
Flaky test	A test that sometimes passes, sometimes fails for no code reason (timing, random data). *Example: Major CI problem — erodes trust in the pipeline. Fix immediately.*
Test pyramid	Design: many unit tests, fewer integration, very few E2E. Optimises speed vs confidence. *Example: More unit tests = faster pipeline with high confidence*

SECURITY SCANNING

SAST	Static Application Security Testing — scans source code for vulnerabilities without running it. *Example: Catches SQL injection, XSS, insecure crypto. Tools: SonarQube, Semgrep*
DAST	Dynamic AST — attacks a running application like a hacker, finding runtime vulnerabilities. *Example: Tools: OWASP ZAP, Burp Suite*
SCA	Software Composition Analysis — scans third-party libraries for known CVEs. *Example: Tools: Snyk, Dependabot, Black Duck*
CVE	Common Vulnerabilities and Exposures — public database of known security bugs with severity scores. *Example: CVE-2021-44228 = Log4Shell, one of the most critical ever discovered*
Shift-left security	Find security issues as early as possible — in developer IDE or CI build, not after deployment. *Example: Cheaper and faster to fix bugs found early in development*
SQL injection	Attacker inserts malicious SQL into an input to manipulate the database. *Example: Prevented by parameterised queries. One of the most common web vulnerabilities*
Container scanning	Scans Docker images for known vulnerable OS packages inside the container. *Example: Tools: Trivy, Grype, Docker Scout*

DEPLOYMENT STRATEGIES

Environment	Isolated copy of the system for a specific purpose: dev → staging → production. *Example: Each has separate data, access controls, and configuration*

Blue/Green deploy	Run two identical production environments. Switch load balancer from Blue (old) to Green (new). *Example: Instant rollback = switch traffic back to Blue*
Canary deployment	Release to a small % of users first (e.g. 5%), watch metrics, gradually increase to 100%. *Example: Limits blast radius of bad releases to a fraction of users*
Rolling deployment	Replace old instances one by one until all run the new version. No downtime required. *Example: Kubernetes does rolling deployments natively out of the box*
Rollback	Revert production to a previous known-good version when a deployment causes errors. *Example: Fast rollback (<2 min) is a sign of a mature CI/CD pipeline*
Feature flag	Deploy code to production but hide behind a toggle. Enable without redeploying. *Example: Tools: LaunchDarkly, Flagsmith. Decouples deploy from release.*
GitOps	Git repo is the single source of truth for infrastructure and deployments. *Example: ArgoCD watches the repo and auto-syncs the cluster to match*
Approval gate	Manual checkpoint requiring a human to click Approve before the next stage runs. *Example: Used before production deploys in Continuous Delivery*
IaC	Infrastructure as Code — define servers, networks, databases in version-controlled files. *Example: Tools: Terraform, AWS CloudFormation, Pulumi*
Kubernetes (K8s)	Open-source container orchestration — manages deploying, scaling, and healing containers. *Example: Runs containers across a cluster of servers automatically*
Helm	Package manager for Kubernetes. Bundles K8s YAML files into reusable charts. *Example: helm upgrade shopeasy ./chart --set image.tag=v2.4.1*
Immutable infra	Never modify servers after deployment — destroy and replace with new images. *Example: Eliminates configuration drift and snowflake servers*

MONITORING & OBSERVABILITY

Observability	Ability to understand what's happening inside a system from its external outputs. *Example: Built on three pillars: Metrics, Logs, and Traces*
Metrics	Numerical measurements over time — CPU %, request rate, error count, latency. *Example: Prometheus collects metrics; Grafana visualises them*
Logs	Time-stamped text records of events emitted by the app. Used for debugging incidents. *Example: Centralised with ELK Stack: Elasticsearch, Logstash, Kibana*
Distributed tracing	Follow a single request across multiple microservices to see where time was spent. *Example: Tools: Jaeger, Zipkin, AWS X-Ray*
Latency (p99)	99th percentile response time — slowest response 1 in 100 users experiences. *Example: Better than averages for detecting performance outliers*
Alerting	Automatically notify engineers when a metric exceeds a threshold. *Example: PagerDuty wakes on-call engineer if error rate > 1% for 5 minutes*
SLO	Service Level Objective — internal reliability target. *Example: SLO: 99.9% of requests respond in <200ms. Violation = incident.*
MTTR	Mean Time To Restore — average time to fix a production incident. A DORA metric. *Example: Elite teams achieve <1 hour MTTR*
On-call	Engineer assigned to be reachable 24/7 to respond to production incidents. *Example: Teams rotate on-call duty weekly or bi-weekly*

DORA Metrics — Measure Pipeline Health

Deployment Frequency	Lead Time for Changes	Change Failure Rate	MTTR
How often you deploy to production	Commit to production time	% of deploys causing incidents	Time to restore after incident
Elite: multiple/day · Low: monthly	Elite: <1 hour · Low: >6 months	Elite: 0-5% · Low: 46-60%	Elite: <1 hour · Low: >6 months

Popular CI/CD Tools

GitHub Actions CI/CD · Cloud · Free tier	**GitLab CI/CD** CI/CD · Built-in · Self-host	**Jenkins** CI/CD · Open source · Self-host
CircleCI CI/CD · Cloud-native · SaaS	**ArgoCD** GitOps CD · Kubernetes-native	**Snyk** SCA + SAST · Security scanning
SonarQube Code quality + SAST	**Trivy** Container + IaC scanning	**Prometheus** Metrics collection · Open source
Grafana Dashboards + alerting	**Kubernetes** Container orchestration	**Terraform** IaC · Multi-cloud

Best Practice Tips

• Fail fast — put cheapest checks (lint, unit tests) first in the pipeline	• Never hardcode secrets — use encrypted CI environment variables
• Pin all dependency versions for reproducible, trustworthy builds	• Every pipeline run must be idempotent — safe to re-run without side effects
• Test in an environment that mirrors production as closely as possible • Use feature flags to decouple deployment from feature release	• Track DORA metrics every sprint to measure and improve pipeline health • Treat your pipeline configuration as production code — review and test it